

Nested if condition : [Nested means one inside another]

If an **'if condition'** is placed inside another **if condition** then it is called Nested if.
In nested if condition, we have one outer if and one inner if condition, the inner if condition **will only execute** when outer if condition returns true.
We can also write if condition inside else block.

Syntax:

```
if(condition) //Outer if condition
{
    if(condition) //inner if condition
    {
        //inner else
    }
}
else //outer else
{
    if(condition) //inner if condition
    {
        //inner else
    }
}
```

```
/*
Find the biggest number among 3 numbers
*/

void main()
{
    int a = Integer.parseInt(IO.readLine("Enter the value of a :"));
    int b = Integer.parseInt(IO.readLine("Enter the value of b :"));
    int c = Integer.parseInt(IO.readLine("Enter the value of c :"));
    int big = 0;

    if(a > b)
    {
        if(a > c)
        {
            big = a;
        }
        else
        {
            big = c;
        }
    }
    else //b is greater than a
    {
        if(b > c)
        {
            big = b;
        }
        else
        {
            big = c;
        }
    }

    IO.println("Biggest number is : "+big);
}
```

Note: In the above program to find out the biggest number among three number we need to take the help of **nested if condition** but the code becomes complex, to reduce the length of the code Logical Operator came into the picture.

Logical OR Conditional Operator:

In Java we have 3 logical Operators

- 1) && (Logical AND Operator)
- 2) || (Logical OR Operator)
- 3) ? : Ternary (shorthand for if-then-else statement)

&&: All the conditions must be true, if the first expression is **false** it will not check right side expressions that is the reason It is short circuit operator

||: Among multiple conditions, at least one condition must be **true**, if the first expression is true it will not check right side expressions that is the reason It is short circuit operator

?: It consists of **three operands**. It is used to evaluate boolean expressions. The operator decides which value will be assigned to the variable. It is used to reduce the size of **if-else condition**.

Note: The && and || operator only works with boolean operand so the following code will not compile.

```
if(5 && 6) //error
{
}

//Logical AND
void main()
{
    int a = 10, b = 20;
    boolean result = (a < b) && (b > 15);
    IO.println(result);
}
```

Biggest number among 3 numbers by using logical AND :

```
void main()
{
    int a =Integer.parseInt(IO.readLine("Enter the value of a :"));
    int b =Integer.parseInt(IO.readLine("Enter the value of b :"));
    int c =Integer.parseInt(IO.readLine("Enter the value of c :"));

    int big=0;

    if(a>b && a>c)
    {
        big = a;
    }
    else if(b > a && b > c)
    {
        big = b;
    }
    else
    {
        big = c;
    }

    IO.println("The big number is : "+big);
}
```

```
//logical OR
void main()
{
    int x = 5, y = 3;

    boolean result = (x > 10) || (y < 5);

    IO.println(result);
}
```

Programs on ternary operator

```
//Ternary Operator
void main()
{
    int x = 5;
    int y = 3;
    int result = 0;
    boolean condition = false;

    result = condition ? x : y;

    IO.println(result);
}
```

Note : If the given condition is true then statement1 (value of x) will assign to result otherwise statement2 (value of y) will assign

```
//Ternary Operator
void main()
{
    int x = Integer.parseInt(IO.readLine("Enter the value of x :"));
    int y = Integer.parseInt(IO.readLine("Enter the value of y :"));
    int max;

    max = x > y ? x : y;

    IO.println("Max number is : "+max);
}
```

Working with Bitwise AND(&), Bitwise OR (|) and Bitwise X-OR (^):

a) Logical Operator works with boolean value only.
Example: IO.println(5 && 6); **//error**

b) Bitwise AND (&), Bitwise OR (|) and Bitwise X-OR (^) can work with number and boolean both types of values.

Example:

```
IO.println(5 & 6); //Bitwise AND (&) //Valid
IO.println(true & true); //Bitwise AND(&) //Valid
```

Program on Bitwise OR Operator

Bitwise OR, where just like **Logical OR** at-least one condition must be true as well as It is known as non-short circuit operator because, If the first condition is true, still It will verify all the right-side condition.

// || VS | [Logical OR VS Bitwise OR]

```
void main()
{
    int z = 5;
    if(++z > 5 || ++z > 6) //6
    {
        z++; //7
    }
    IO.println(z); //7

    IO.println(".....");

    z = 5;
    if(++z > 5 | ++z > 6) //7
    {
        z++; //8
    }
    IO.println(z); //8
}
```

Program on Bitwise AND (&)

```
void main()
{
    int z = 5;
    if(++z > 6 & ++z > 6)
    {
        IO.println("Inside If");
        z++;
    }
    IO.println(z);
}
```

Bitwise X-OR :

//Bitwise X-OR (^) [Same input output is false]

```
void main()
{
    IO.println(true ^ true); //false
    IO.println(true ^ false); //true
    IO.println(false ^ true); //true
    IO.println(false ^ false); //false
}
```

Working with numeric values :

//Bitwise AND, OR, X-OR

```
void main()
{
    IO.println(5 & 6); //4
    IO.println(5 | 6); //7
    IO.println(5 ^ 6); //3
}
```

Binary of 5 : 1 0 1

Binary of 6 : 1 1 0

5 & 6 1 0 0 (4)

Binary of 5 : 1 0 1

Binary of 6 : 1 1 0

5 | 6 1 1 1 (7)

Binary of 5 : 1 0 1

Binary of 6 : 1 1 0

5 ^ 6 0 1 1 (3)

We will get the result using truth table but not using binary calculation

Truth table for AND, OR and X-OR Operator

AND (A & B)

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

OR (A + B)

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

X-OR (A ⊕ B)

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Same Input output is 0

Bitwise Complement Operator (~)

* It will not work with boolean type.

Example :
//Bitwise complement operator
void main()
{
 IO.println(~ false); //error
}

Formula :
~ x = - (x + 1)

* It is used to convert numeric value from positive to negative and negative to positive.

Formula :
~ x = - (x + 1) [Formula for finding complement number]

//Bitwise complement operator ~x = -(x + 1)

```
void main()
{
    IO.println(~ 5); // = -(5 + 1) -6
    IO.println(~ -6); // = -(-6 + 1) 5
}
```

Bitwise Shift Operators in Java

It will work with two operands

It is used to work at the binary (bit) level, not on decimal values.

It is used to shift the bits of an integer value to the left (<<) or right (>>).

They are mainly used for fast arithmetic operations at bit level.

Left shift Operator formula: x<<n = x * 2ⁿ

Right shift Operator formula: x>>n = x / 2ⁿ

//Bitwise Shift operator

```
void main()
{
    IO.println(9<<3); //left shift 9 * 2X2X2 = 72
    IO.println(9>>3); //right shift 9 / 8

    //IO.println(9.2>>3); //Invalid
}
```

Formula :
x << n = x * 2ⁿ
x >> n = x / 2ⁿ

instance of operator we will learn in Dynamic Polymorphism